Crank-Nicolson Method for Usadel Evolution

Numerical Stability Analysis

May 7, 2026

1 Introduction

The current explicit time-stepping scheme for the Usadel equation exhibits exponential instability, amplifying τ_3 errors by a factor of ~ 2 per timestep. This document explains the Crank-Nicolson method and its application to structure-preserving evolution of Green's functions.

2 Crank-Nicolson Method: Simple Example

2.1 Standard Heat Equation

Consider the one-dimensional heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}. \tag{1}$$

Forward Euler (Explicit, Unstable). Discretize time derivative using forward difference:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \alpha \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2}. \tag{2}$$

Rearranging:

$$u_j^{n+1} = u_j^n + r(u_{j+1}^n - 2u_j^n + u_{j-1}^n), \tag{3}$$

where $r = \alpha \Delta t/(\Delta x)^2$.

Stability: Requires $r \leq 1/2$ (conditional stability). For $r > 1/2$, exponential growth occurs.

Crank-Nicolson (Implicit, Unconditionally Stable). Average the spatial derivative at times n and $n+1$:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \frac{\alpha}{2}\left[\left.\frac{\partial^2 u}{\partial x^2}\right|_n + \left.\frac{\partial^2 u}{\partial x^2}\right|_{n+1}\right]. \tag{4}$$

Explicitly:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \frac{\alpha}{2(\Delta x)^2}\left[(u_{j+1}^n - 2u_j^n + u_{j-1}^n) + (u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1})\right]. \tag{5}$$

Rearranging to collect $n+1$ terms on left, n terms on right:

$$-ru_{j-1}^{n+1} + 2(1+r)u_j^{n+1} - ru_{j+1}^{n+1} = ru_{j-1}^n + 2(1-r)u_j^n + ru_{j+1}^n. \tag{6}$$

This is a **tridiagonal linear system** $A\mathbf{u}^{n+1} = B\mathbf{u}^n$, solved at each timestep.

Stability: Unconditionally stable for all $r > 0$ (no timestep restriction from stability, only accuracy).

Accuracy: Second-order in both space and time: error $\sim O(\Delta x^2, \Delta t^2)$.

2.2 Simple ODE Example

For ordinary differential equation:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0, \tag{7}$$

Forward Euler:

$$y^{n+1} = y^n + \Delta t\, f(t_n, y^n). \tag{8}$$

Crank-Nicolson: Evaluate f at the midpoint by averaging:

$$y^{n+1} = y^n + \frac{\Delta t}{2}\left[f(t_n, y^n) + f(t_{n+1}, y^{n+1})\right]. \tag{9}$$

This is **implicit**: y^{n+1} appears on both sides. Requires solving (typically via Newton iteration):

$$y^{n+1} - y^n - \frac{\Delta t}{2} f(t_{n+1}, y^{n+1}) = \frac{\Delta t}{2} f(t_n, y^n). \tag{10}$$

2.3 Unitary Evolution Example

Consider evolution under time-dependent Hamiltonian:

$$\frac{d}{dt}\psi(t) = -iH(t)\psi(t). \tag{11}$$

Forward Euler:

$$\psi^{n+1} = \psi^n - i\Delta t\, H(t_n)\psi^n = [1 - i\Delta t H(t_n)]\psi^n. \tag{12}$$

Problem: Operator $[1 - i\Delta t H]$ is NOT unitary, even if H is Hermitian. Norm $\|\psi\|$ can grow or shrink.

Crank-Nicolson:

$$\psi^{n+1} = \psi^n - \frac{i\Delta t}{2}\left[H(t_n)\psi^n + H(t_{n+1})\psi^{n+1}\right]. \tag{13}$$

Rearranging:

$$\left[1 + \frac{i\Delta t}{2}H(t_{n+1})\right]\psi^{n+1} = \left[1 - \frac{i\Delta t}{2}H(t_n)\right]\psi^n. \tag{14}$$

Unitarity preservation: The propagator is:

$$U_{\text{CN}} = \left[1 + \frac{i\Delta t}{2}H\right]^{-1}\left[1 - \frac{i\Delta t}{2}H\right]. \tag{15}$$

For Hermitian H, this is a Cayley transform: $U_{\text{CN}}^\dagger U_{\text{CN}} = 1$ (exactly unitary, even for finite Δt).

Key advantage: Preserves geometric structure (unitarity, norm conservation) at discrete level.

3 Crank-Nicolson for Usadel Equation

3.1 Current Scheme (Explicit, Unstable)

Your discrete evolution for $\hat{g}'^R(t,t')$ is:

$$\hat{g}'^R(t+\Delta t,t') = \hat{U}_L(\Delta t;t)\Bigg[- \hat{\tau}_3\left(\hat{g}'^R(t,t'+\Delta t)\hat{U}_R^{-1}(\Delta t;t') - \hat{g}'^R(t,t')\right)\hat{\tau}_3 + \hat{g}'^R(t,t') + \delta_{tt'}\hat{g}'^R(t,t)\Bigg]. \tag{16}$$

This uses:

- **Forward derivative in t:** $\partial_t \hat{g} \approx [\hat{g}(t+\Delta t, t') - \hat{g}(t,t')]/\Delta t$
- **Backward derivative in t':** $\partial_{t'} \hat{g} \approx [\hat{g}(t, t'+\Delta t) - \hat{g}(t,t')]/\Delta t$ (via shift)

Problem: This asymmetric stencil breaks time-translation invariance. The term $\hat{g}+\hat{\tau}_3\hat{g}\hat{\tau}_3$ amplifies τ_0, τ_3 components by factor 2 (see Pauli algebra: $\hat{\tau}_3(\tau_3)\hat{\tau}_3 = +\tau_3$, doubling instead of canceling).

3.2 Crank-Nicolson Formulation

Strategy. Average the evolution operator between times t and $t+\Delta t$:

$$\frac{\partial \hat{g}'^R}{\partial t}(t,t') \approx \frac{1}{2}\left[\mathcal{L}_t[\hat{g}'^R](t,t') + \mathcal{L}_t[\hat{g}'^R](t+\Delta t, t')\right], \tag{17}$$

where $\mathcal{L}_t$ is the Usadel evolution operator (derived from continuous equation).

Detailed construction. The continuous Usadel equation (after renormalization) gives:

$$\partial_t \hat{g}'^R(t,t') = i\hat{\tau}_3[\hat{\Delta}(t) - \hat{g}'^R(t,t)\delta(t-t')] - i[\hat{\Delta}(t) - \hat{g}'^R(t,t)\delta(t-t')]\hat{\tau}_3 + \mathcal{D}_{t'}\hat{g}'^R, \tag{18}$$

where $\mathcal{D}_{t'}$ represents the derivative term in t'.

For Crank-Nicolson, discretize as:

$$\frac{\hat{g}'^R(t+\Delta t, t') - \hat{g}'^R(t,t')}{\Delta t} = \frac{1}{2}\left[\mathcal{L}_t[\hat{g}'^R](t,t') + \mathcal{L}_t[\hat{g}'^R](t+\Delta t, t')\right]. \tag{19}$$

Off-diagonal evolution ($t' \neq t$, $\delta_{tt'} = 0$). Using the transformation $\hat{g}'^R = \hat{U}_L \hat{h}^R \hat{U}_R$ and integrating over $[t, t+\Delta t]$, the Crank-Nicolson version becomes:

$$\begin{aligned} &\hat{g}'^R(t+\Delta t, t') - \hat{U}_L(\Delta t; t)\hat{g}'^R(t,t') \\ &= -\frac{1}{2}\Bigg[\hat{U}_L(\Delta t; t)\hat{\tau}_3\left(\hat{g}'^R(t, t'+\Delta t)\hat{U}_R^{-1}(\Delta t; t') - \hat{g}'^R(t,t')\right)\hat{\tau}_3 \\ &\qquad + \hat{\tau}_3\left(\hat{g}'^R(t+\Delta t, t'+\Delta t)\hat{U}_R^{-1}(\Delta t; t') - \hat{g}'^R(t+\Delta t, t')\right)\hat{\tau}_3\Bigg]. \end{aligned} \tag{20}$$

Rearranging to separate known (time t) from unknown (time $t + \Delta t$) terms:

$$\begin{aligned}
&\left[1 + \frac{1}{2}\hat{\tau}_3 \cdot \hat{U}_R^{-1}(\Delta t; t')\hat{\tau}_3\right] \hat{g}'^R(t + \Delta t, t') \\
&= \hat{U}_L(\Delta t; t) \left[1 - \frac{1}{2}\hat{\tau}_3 \hat{U}_R^{-1}(\Delta t; t')\hat{\tau}_3\right] \hat{g}'^R(t, t') \\
&\quad + \frac{1}{2}\hat{\tau}_3 \left[\hat{g}'^R(t, t' + \Delta t) + \hat{g}'^R(t + \Delta t, t' + \Delta t)\hat{U}_R^{-1}(\Delta t; t')\right] \hat{\tau}_3.
\end{aligned} \tag{21}$$

Structure: This is a linear system:

$$\mathcal{A}\,\mathbf{g}^{n+1} = \mathcal{B}\,\mathbf{g}^n, \tag{22}$$

where $\mathcal{A}$ and $\mathcal{B}$ are operators involving $\hat{\tau}_3$ and unitary propagators.

3.3 Advantages of Crank-Nicolson for Usadel

1. **Time-translation invariance preservation.** The symmetric averaging ensures that in equilibrium, $\hat{g}(t + \Delta t, t) = \hat{g}(t, t - \Delta t)$ holds to $O(\Delta t^2)$.

2. **Algebraic stability.** The term $\hat{g} + \hat{\tau}_3\hat{g}\hat{\tau}_3$ appears symmetrically on both sides, preventing amplification:

   $$\text{LHS: } (1 + \text{operator})\hat{g}^{n+1}, \quad \text{RHS: } (1 - \text{operator})\hat{g}^n. \tag{23}$$

   This Cayley-type structure bounds the propagator.

3. **Second-order accuracy.** Errors scale as $O(\Delta t^2)$ instead of $O(\Delta t)$.

4. **Unconditional stability.** No timestep restriction from stability (unlike explicit Euler which requires $\Delta t < \Delta t_{\text{crit}}$).

3.4 Computational Cost

Explicit scheme:

- Direct update: $\hat{g}^{n+1} = f(\hat{g}^n)$
- Cost per timestep: $O(N_t^2 \times N_{\text{Pauli}})$ (matrix operations)
- No linear solve required

Crank-Nicolson:

- Implicit solve: $\mathcal{A}\hat{g}^{n+1} = \mathcal{B}\hat{g}^n$
- Cost per timestep: $O(N_t^3)$ (direct solve) or $O(N_t^2 \log N_t)$ (iterative)
- For Nambu-Keldysh structure: $4\times$ cost (4 Pauli components)

Tradeoff: Crank-Nicolson is $\sim$ 2-5$\times$ more expensive per step, but allows larger timesteps (factor 5-10) due to unconditional stability. Net speedup possible.

3.5 Implementation Strategy

Diagonal elements ($t' = t$). For equal-time Green's function, use the simplified form from paper Eq. 2318:

$$\hat{g}'^R(t + \Delta t, t) = \hat{U}_L(\Delta t; t)\hat{g}'^R(t, t). \tag{24}$$

This is already exact (no discretization error in derivative), so keep as-is.

Off-diagonal elements ($t' < t$). Implement Crank-Nicolson as described above. For each new time slice t_{n+1}:

1. **Form RHS:** Compute known terms involving $\hat{g}^n$ at time t_n
2. **Solve linear system:** Invert operator $\mathcal{A}$ to obtain $\hat{g}^{n+1}$
3. **Update grid:** Store new row $\hat{g}'^R(t_{n+1}, t_j)$ for all $j \leq n + 1$

Linear solver options. Option 1: Direct solve (small grids, $N_t <$ 500)

- Use `numpy.linalg.solve` or `scipy.linalg.solve`
- Exploit sparsity if operator $\mathcal{A}$ is banded

Option 2: Iterative solve (large grids, $N_t > 500$)

- Use `scipy.sparse.linalg.gmres` or `bicgstab`
- Precondition with approximate inverse $\mathcal{A}^{-1} \approx [1 + \text{operator}]^{-1}$
- Typically converges in $\sim$ 5-10 iterations

Option 3: Predictor-corrector (hybrid)

- **Predictor:** Explicit step to estimate $\hat{g}^{n+1}_{\text{guess}}$
- **Corrector:** Crank-Nicolson iteration starting from guess:

$$\hat{g}^{n+1}_{k+1} = \mathcal{A}^{-1}\left[\mathcal{B}\hat{g}^{n} + \text{coupling terms}(\hat{g}^{n+1}_{k})\right]. \tag{25}$$

- Usually converges in 1-2 iterations

4 Practical Modifications for Your Code

4.1 Minimal Change (Projection Method)

As a quick fix without rewriting the solver, add a projection step after each explicit update:

$$\hat{g}'^{R}_{\text{corrected}} = \hat{g}'^{R}_{\text{explicit}} - \frac{1}{2}(\hat{\tau}_0\hat{g}\hat{\tau}_0 + \hat{\tau}_3\hat{g}\hat{\tau}_3), \tag{26}$$

where the factor $1/2$ cancels the factor-2 amplification. This is not Crank-Nicolson, but stabilizes the scheme.

4.2 Full Crank-Nicolson Implementation

Replace `_compute_new_gr_row` function:

```
def _compute_new_gr_row_CN(self, state, external_field=None):
    """
    Crank-Nicolson evolution for g'^R(t,t').
    Solves implicit linear system at each timestep.
    """
    # Extract current state
    gr_grid = state['gr']
    gap_history = state['gap_history']

    # Build coefficient matrices
    A_operator = self._build_CN_LHS_operator(gap_history[-1])
    B_operator = self._build_CN_RHS_operator(gap_history[-1])

    # Form RHS: B * g^n + coupling terms
    rhs = B_operator @ gr_last_row + coupling_terms(...)

    # Solve linear system: A * g^{n+1} = rhs
```

```
gr_new = scipy.sparse.linalg.gmres(A_operator, rhs)[0]

return gr_new, gr_diagonal_new
```

4.3 Testing and Validation

After implementing Crank-Nicolson, verify:

1. **Equilibrium stability:** Evolve equilibrium initial condition for many timesteps. Check:
$$\max_{ij} |\hat{g}'^R(t_i, t_j) - \hat{g}'^R_{\text{eq}}(t_i - t_j)| < 10^{-8}. \tag{27}$$

2. **Time-translation invariance:** For all i, j:
$$\|\hat{g}'^R(t_{i+1}, t_j) - \hat{g}'^R(t_i, t_{j-1})\| < C\Delta t^2, \tag{28}$$
where C is a constant (2nd-order convergence).

3. τ_3 **component stability:** Monitor $\max|\tau_3|$ over time. Should remain bounded (not grow exponentially).

4. **Normalization:** Check $\{\hat{g}'^R(t,t), \hat{\tau}_3\} = 0$ at all times.

5. **Convergence test:** Run with Δt, $\Delta t/2$, $\Delta t/4$ and verify:
$$\text{Error}(\Delta t/2) \approx \text{Error}(\Delta t)/4 \quad \text{(2nd order)}. \tag{29}$$

5 Conclusion

The current explicit scheme has a fundamental instability that amplifies τ_3 errors by factor ~ 2 per timestep, independent of Δt. This is due to the asymmetric time-differencing which breaks time-translation invariance.

Crank-Nicolson fixes this by:

- Using symmetric (midpoint) averaging of evolution operator
- Preserving geometric structure (unitarity, time-translation symmetry)
- Providing unconditional stability and 2nd-order accuracy

The cost is an implicit solve at each timestep, but this is offset by the ability to use larger timesteps and guaranteed stability.

For immediate results, use the projection method as a temporary fix. For publication-quality numerics, implement full Crank-Nicolson.